ReproBot: A Multi-Agent System for Automated Scientific Paper Replication

Aleyna Kütük
aleynakutuk10@gmail.com

Engincan Varan
engincanvaran@gmail.com

May 24, 2026

Abstract

Reproducing results from machine learning research papers is a critical yet time-consuming task that demands both deep domain knowledge and substantial engineering effort. We propose **ReproBot**, a multi-agent AI pipeline that autonomously reads a scientific paper in PDF format, extracts the proposed method and key claims using vision and language models, generates executable experiment code, runs it in a sandboxed environment, and produces a structured replication report comparing reproduced metrics against the paper's stated numbers. The system combines agentic reasoning, visual document understanding, and generative code synthesis to lower the barrier to scientific reproducibility.

Objectives

- Parse ML papers (text, figures, tables) using vision-language models and extract structured method descriptions, key claims, and hyperparameters.
- Generate runnable PyTorch/HuggingFace experiment scripts from extracted method descriptions without human intervention.
- Execute generated code safely in an isolated Docker sandbox and capture evaluation metrics.
- Compare reproduced results against paper claims and trigger targeted code revisions through an iterative Critic–Coder feedback loop.
- Produce a structured replication report with a claim-by-claim comparison and gap analysis.
- Evaluate the full pipeline on a benchmark of 20 image-classification papers, measuring replication rate and the impact of the refinement loop.

Methodology

ReproBot is structured as a four-agent pipeline coordinated by a central Orchestrator that manages a shared memory state. The **Reader** agent ingests a PDF using `pdfplumber` and a vision-language model to extract method summaries, claims, datasets, and hyperparameters. The **Coder** agent translates this structured output into a self-contained HuggingFace `Trainer`-based training script. The **Runner** agent executes the script inside a Docker container, captures metrics and error traces, and writes results back to shared memory. The **Critic** agent compares reproduced metrics against the paper's stated numbers, produces a structured verdict, and — if needed — feeds targeted feedback to the Coder to trigger a rewrite. The Orchestrator drives this loop until results meet a confidence threshold or the retry budget is exhausted, then hands

off to a report generator that produces a structured Markdown document with the final code, results table, and analysis.

Required Skillset and Tools

- **ML fundamentals:** CNNs, Vision Transformers, supervised learning, evaluation metrics.
- **Programming:** Python, PyTorch, HuggingFace `transformers` and `datasets`.
- **LLM / VLM:** Prompt engineering, OpenAI or open-source API usage, vision-language models (LLaVA / GPT-4V).
- **Infrastructure:** Docker, Unix shell scripting, Git.
- **Libraries:** LangChain (agent framework), `pdfplumber`, `pdf2image`, Gradio (demo), Weights & Biases (experiment tracking).

Proposed Timeline

- **Month 1:** Reader agent — PDF ingestion, vision-language parsing, structured information extraction.
- **Month 2:** Coder and Runner agents — code generation, Docker sandbox, Orchestrator skeleton with basic retry.
- **Month 3:** Critic agent — claim verification, iterative refinement loop, report generation.
- **Month 4:** Evaluation on 20-paper benchmark, ablation study, Gradio demo, and final project report.